

BootIT

Ai Ting Lee

Based on slides by Niek Janssen

-- Based on slides by Sebastian Mateos Nicolajsen

---- Based on slides by Claus Brabrand

Welcome
Back!!!

Expressions

- **Literal Value(42)**
- **Variable (x)**
- **Comparison** between expressions
 - $\text{Exp} == \text{Exp}$
 - $\text{Exp} != \text{Exp}$
 - $\text{Exp} > \text{Exp}$
 - $\text{Exp} < \text{Exp}$
 - $\text{Exp} >= \text{Exp}$
 - $\text{Exp} <= \text{Exp}$
- **Operation** between expressions
 - $\text{Exp} + \text{Exp}$
 - $\text{Exp} - \text{Exp}$
 - $\text{Exp} * \text{Exp}$
 - Exp / Exp
 - $\text{Exp} \% \text{Exp}$
- **Logical Operators**
 - $\text{Exp} \&\& \text{Exp}$
 - $! \text{Exp}$
 - $\text{Exp} || \text{Exp}$

Statements

- **Assignment**
 - Variable = Expressions
 - Variable = Constant
- **Conditional**
 - if(expression) { statement }
 - if(expression) { statement } else { statement }
- **Loops**
 - while(expression) { statement }
 - for(initialization, expression, iteration) { statement }

And now...

FOR SOMETHING
COMPLETELY
DIFFERENT

Computational Thinking

Who will read your code? Readability for whom?

For Human: Others, Future you, LLM

- Meaningful Naming

```
double celsius = 25.0;  
double fahrenheit = (celsius * 1.8) + 32;
```

≡

```
double a = 25.0;  
double b = (a * 1.8) + 32;
```

- Structural Indentation

```
static String daypart(int time) {  
    if(time == 0) {  
        return "midnight";  
    } else if(time < 6) {  
        return "night";  
    } else if(time < 12) {  
        return "morning";  
    } else if(time < 18) {  
        return "afternoon";  
    } else {  
        return "evening";  
    }  
}
```

≡

```
static String daypart(int time) {  
    if(time == 0) {        return "midnight";  
    } else if(time < 6) return "night";}  
    else if(time < 12) {    return "morning";  
    } else if(time < 18) {  
        return "afternoon";  
    } else {  
        return "evening";}}}
```

Languages

- **Syntax** determines structure: What can I write: where, how, and in what order?
- **Semantics** determines meaning: What does this actually do?
- Humans use context to interpret language:
 - **Syntax** OK, **Semantics** nonsense: *"Colorless green ideas sleep furiously."*
(Humans try to interpret the hidden intent.)
 - **Syntax** wrong, **Semantics** clear: *"Me goes market yesterday buyed 3 apple."*
(Computers crash with Syntax Error, but humans understand instantly.)
- Computers check Syntax first, then evaluate Semantics:
 - **Syntax** valid, **Semantics** wrong: Code compiles and runs fine, but give wrong results.

```
if (true) {  
    System.out.println("This is not true");  
} else {  
    System.out.println("This is true");  
}
```

Error Tolerance

- Humans
 - Letters & Numbers and a few symbols
 - . , ; ! ? ' " - () &
 - Minor mistakes aren't a problem!
- Computers
 - Letters & Numbers and a LOT of symbols
 - . , ; ! ? ' " - () < > [] { } + * / % \ | \$ # & = _ ^
 - 1 misplaced semi-colon and nothing works



Ambiguity



- Humans
 - Very Ambiguous
- Computers
 - As unambiguous as possible

Interpretation A

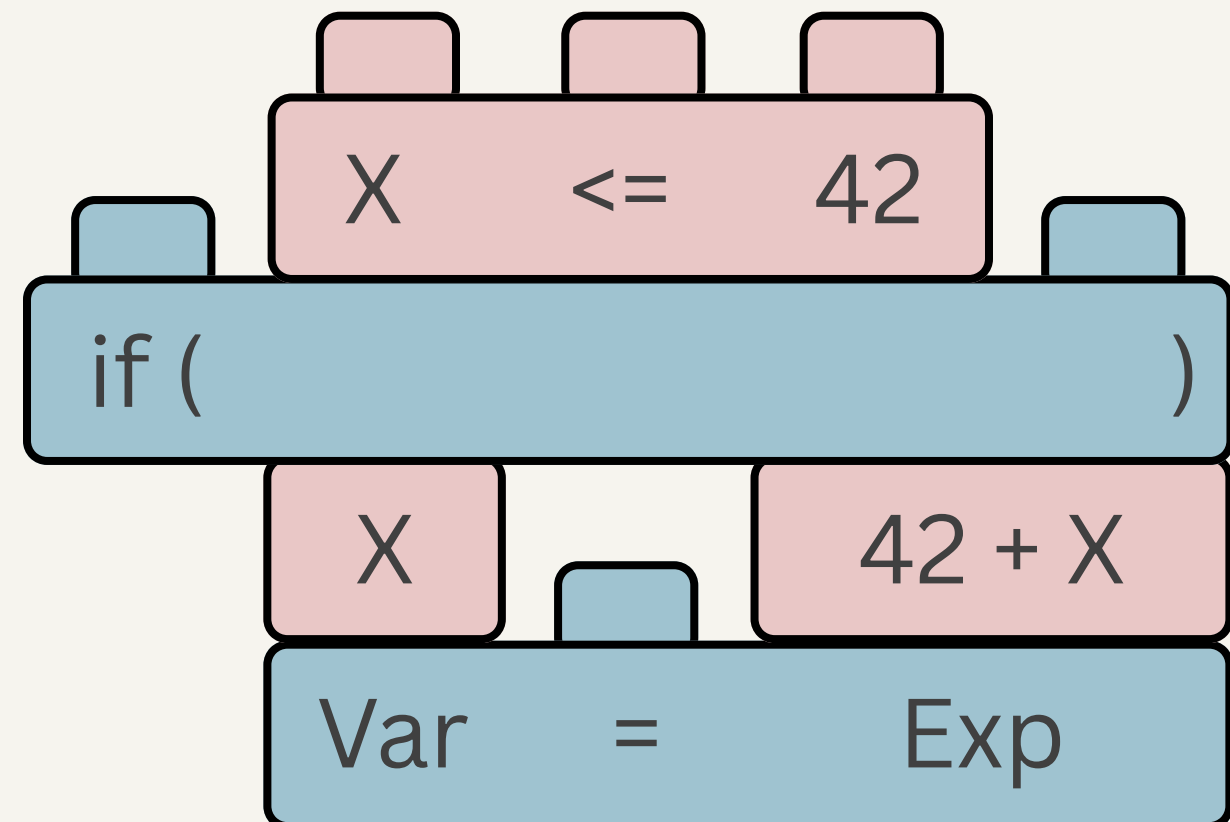
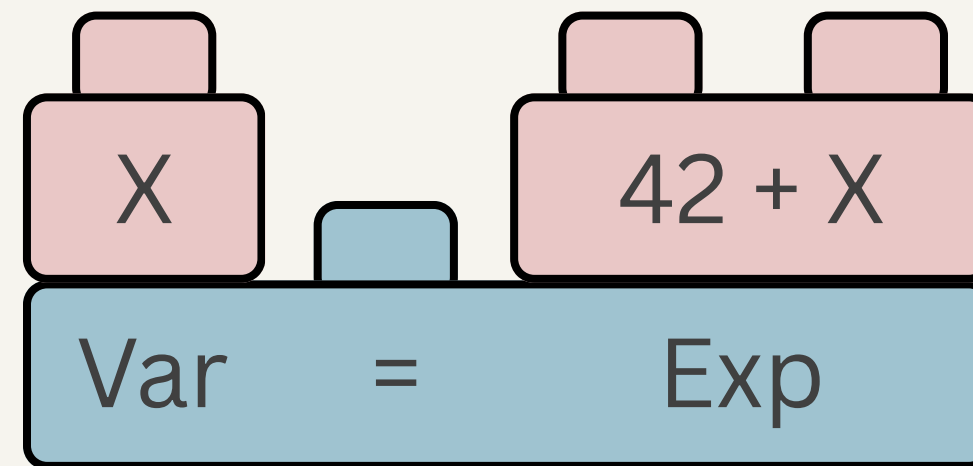
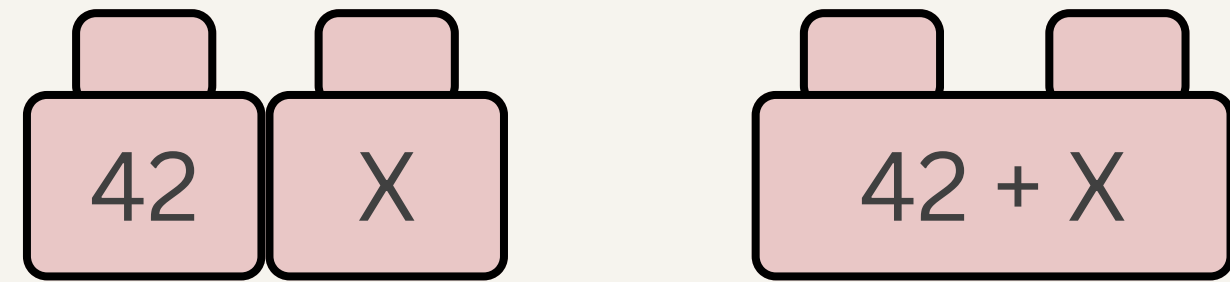
```
int milkToBuy = 1;  
  
if (thereIsEgg) {  
    int eggToBuy = 6;  
}  
// Result : 1 milk + 6 eggs
```

Interpretation B

```
int milkToBuy = 1;  
  
if (thereIsEgg) {  
    milkToBuy = 6;  
}  
// Result : 6 milk
```

JAVA Grammar

- **EXP**ression
 - **CONST**ant (42)
 - **VARI**able (x)
 - Expression + Expression
- **ST**ate**M**ent(**S**)
 - VAR = EXP
 - if(EXP){ STM }
 - if(EXP){ STM } else { STM }
 - while (EXP){ STM }
 - for(EXP; EXP; EXP){ STM }
 - STM + STM



Turing Complete

Turing Completeness describes a machine or a programming language that can execute any computational algorithm.

1. **State:** Ability to store & change data (constant/ variable)
2. **Decision:** Ability to make choices (if / else)
3. **Loop:** Ability to repeat tasks (while/for)

Theoretically Java can be converted into any other Turing Complete Language. But it would be cumbersome and inefficient.

Is Turing Completeness Enough?

- **Computer Speed** (Hardware Limits)
 - Different code can solve the same problem, but in different speeds.
 - Hardware has physical & resource limits.
- **Program Complexity** (Human Limits)
 - As real-world problems scale, brute-force logic breaks down.
 - We need higher-level structures to model domain logic accurately.

Assuming each evaluation (to visit or skip a city) takes 0.001s:



$2^1 = 2$		0,002 sec.
$2^2 = 4$		0,004 sec.
$2^3 = 8$		0,008 sec.
$2^4 = 16$		0,016 sec.
$2^5 = 32$		0,032 sec.
$2^6 = 64$		0,064 sec.
$2^7 = 128$		0,128 sec.
$2^8 = 256$		0,256 sec.
$2^9 = 512$		0,512 sec.
$2^{10} = 1.024$	thousand	1,024 = 1 sec
$2^{20} = 1.048.576$	million	1.024,576 = 17 min
$2^{30} = 1.073.741.824$	billion	1.073.741,824 = 12 days
$2^{40} = 1.099.511.627.776$	trillion	1.099.511.627,776 = 35 years
$2^{50} = 1.125.899.906.842.624$		1.125.899.906.842,624 = 35702 years

Object Oriented Programming

- **Class (Blueprint):** Defines every kart must have Body, Tire, and Glider.
- **Object (Instance):** The actual kart created with your choices.

Porsche 911 GT3

Wheelbase 2457mm
Length 4570mm

Double Wishbone Front Suspension
Independent Multilink Rear Suspension

Width 1852mm

Height 1279mm

255/35 ZR 20" Front Tires
410mm Carbon Ceramic Front Disc Brakes

315/30 ZR 21" Rear Tires
390mm Carbon Ceramic Rear Disc Brakes

Make	Porsche	Engine Specs	Drivetrain	
Model	911 GT3	Displacement 3996cc	Layout Rear Engine, RWD	
Model Year	2025	Configuration Flat-six	Gearbox 6 speed Manual or 7-speed PDK Auto	
Chassis Code	992.2	Induction NA		
Body	2-door, coupe	Power 510hp @ 8500rpm	Curb Weight 1439kg	
Manufactured at Germany	Torque 450nm @ 6250rpm			
	0-100 km/h 3.4 sec			
	Top Speed 311 km/h			

Badwagon

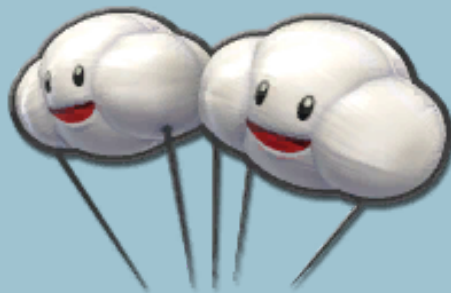
OK

Model of Reality

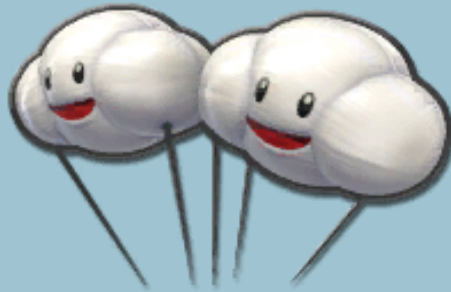


From real-world concepts to computer models

Concept



Select Badwagon



Computer Model

```
String body = "Bloom";  
String tire = "Wood";  
String glider = "Cloud";
```

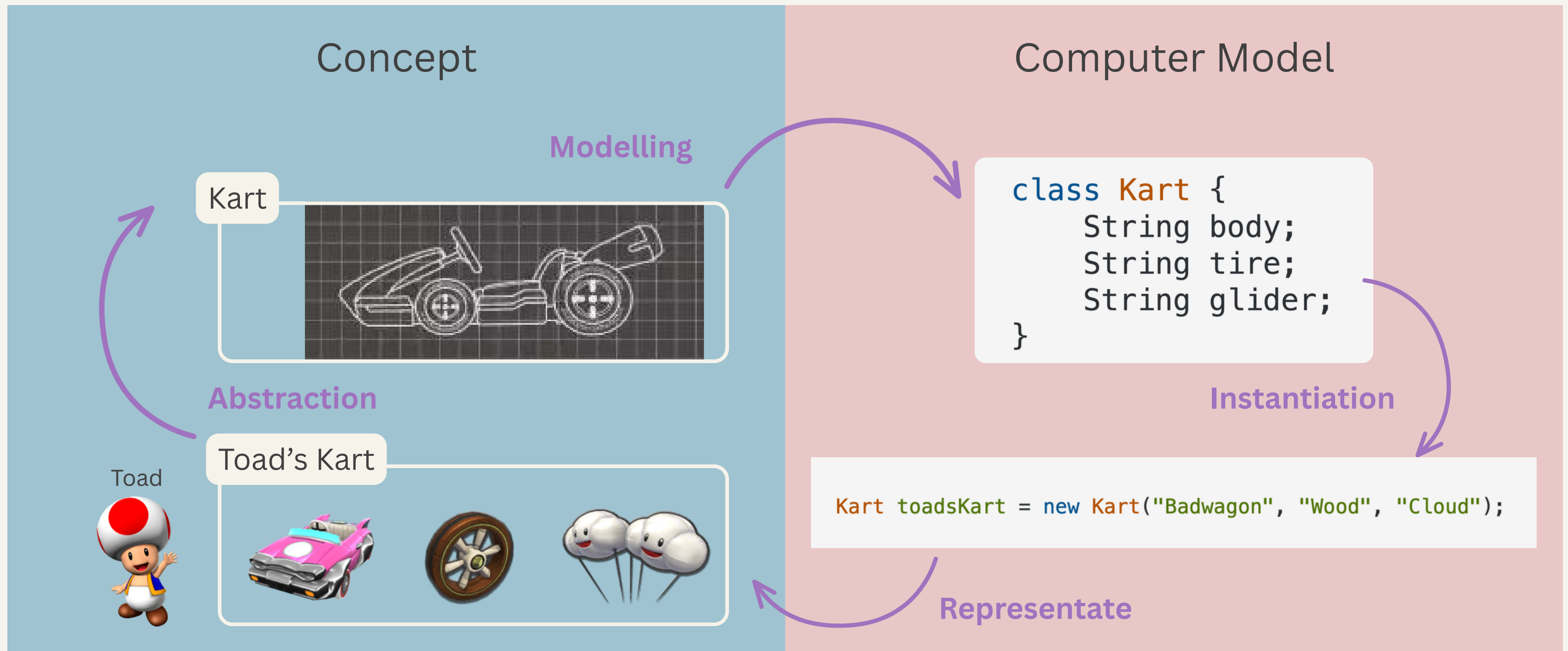


```
body = "Badwagon";
```

```
String body = "Badwagon";  
String tire = "Wood";  
String glider = "Cloud";
```

Modelling

Transformation of Representation of Information



Kart.java

0. Create a new file: **Kart.java**

1. **Class:** Create a new file: Kart.java

```
class Kart {  
    //Empty now  
}
```

2. **Fields:** In the model a Kart has 3 string values - a body, tire, and a glider.

```
class Kart {  
    String body;  
    String tire;  
    String glider;  
}
```


3. Constructor:

- In Java, a class needs a Constructor
- A special **function** with the **same name** used to make a new instance of the class

```
class Kart {  
    String body;  
    String tire;  
    String glider;  
  
    Kart(String b, String t, String g) {  
        body = b;  
        tire = t;  
        glider = g;  
    }  
}
```

Note: In the constructor, we assign the function parameters (right side) to the fields (left side).

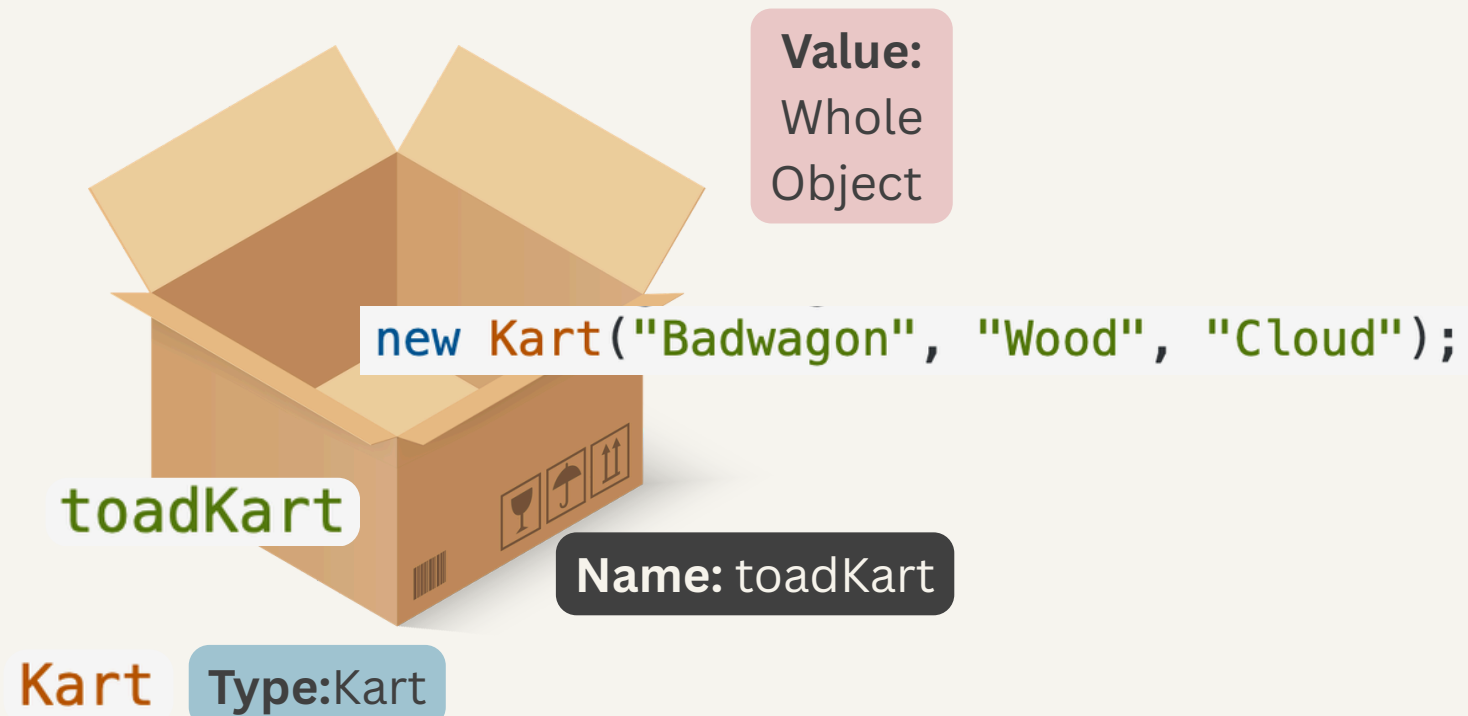
4. Method: Now we can add some functionality

```
class Kart {  
    String body;  
    String tire;  
    String glider;  
  
    Kart(String b, String t, String g) {  
        body = b;  
        tire = t;  
        glider = g;  
    }  
  
    void displayInfo() {  
        System.out.println("Kart Configuration:");  
        System.out.println("- Body: " + body);  
        System.out.println("- Tire: " + tire);  
        System.out.println("- Glider: " + glider);  
    }  
}
```

Main.java + Kart.java

5. Instantiate: Use the class blueprint (Kart.java) to create an actual object

```
public class Main {  
    public static void main(String[] args) {  
        Kart toadKart = new Kart("Badwagon", "Wood", "Cloud");  
    }  
}
```



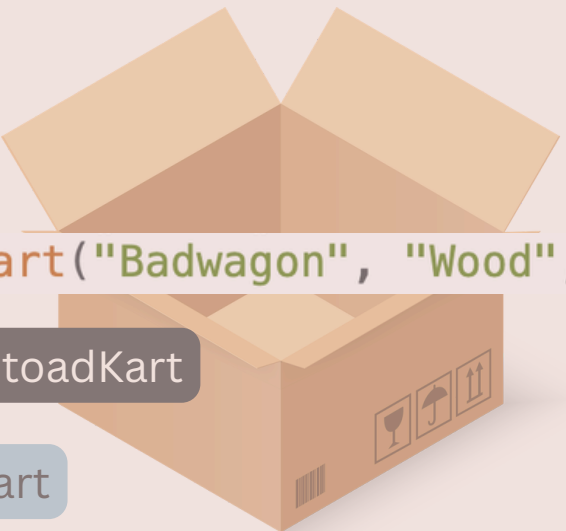
```
class Kart {  
    String body;  
    String tire;  
    String glider;  
  
    Kart(String b, String t, String g) {  
        body = b;  
        tire = t;  
        glider = g;  
    }  
  
    void displayInfo() {  
        System.out.println("Kart Configuration:");  
        System.out.println("- Body: " + body);  
        System.out.println("- Tire: " + tire);  
        System.out.println("- Glider: " + glider);  
    }  
}
```

Main.java + Kart.java

5. **Call:** Refer to the instances' name to call the method.

```
public class Main {  
    public static void main(String[] args) {  
        Kart toadKart = new Kart("Badwagon", "Wood", "Cloud");  
        toadKart.displayInfo();  
    }  
}
```

```
class Kart {  
    String body;  
    String tire;  
    String glider;  
  
    Kart(String b, String t, String g) {  
        body = b;  
        tire = t;  
        glider = g;  
    }  
  
    void displayInfo() {  
        System.out.println("Kart Configuration:");  
        System.out.println("- Body: " + body);  
        System.out.println("- Tire: " + tire);  
        System.out.println("- Glider: " + glider);  
    }  
}
```



```
new Kart("Badwagon", "Wood", "Cloud");
```

Name: toadKart

Type: Kart

toadKart.displayInfo()

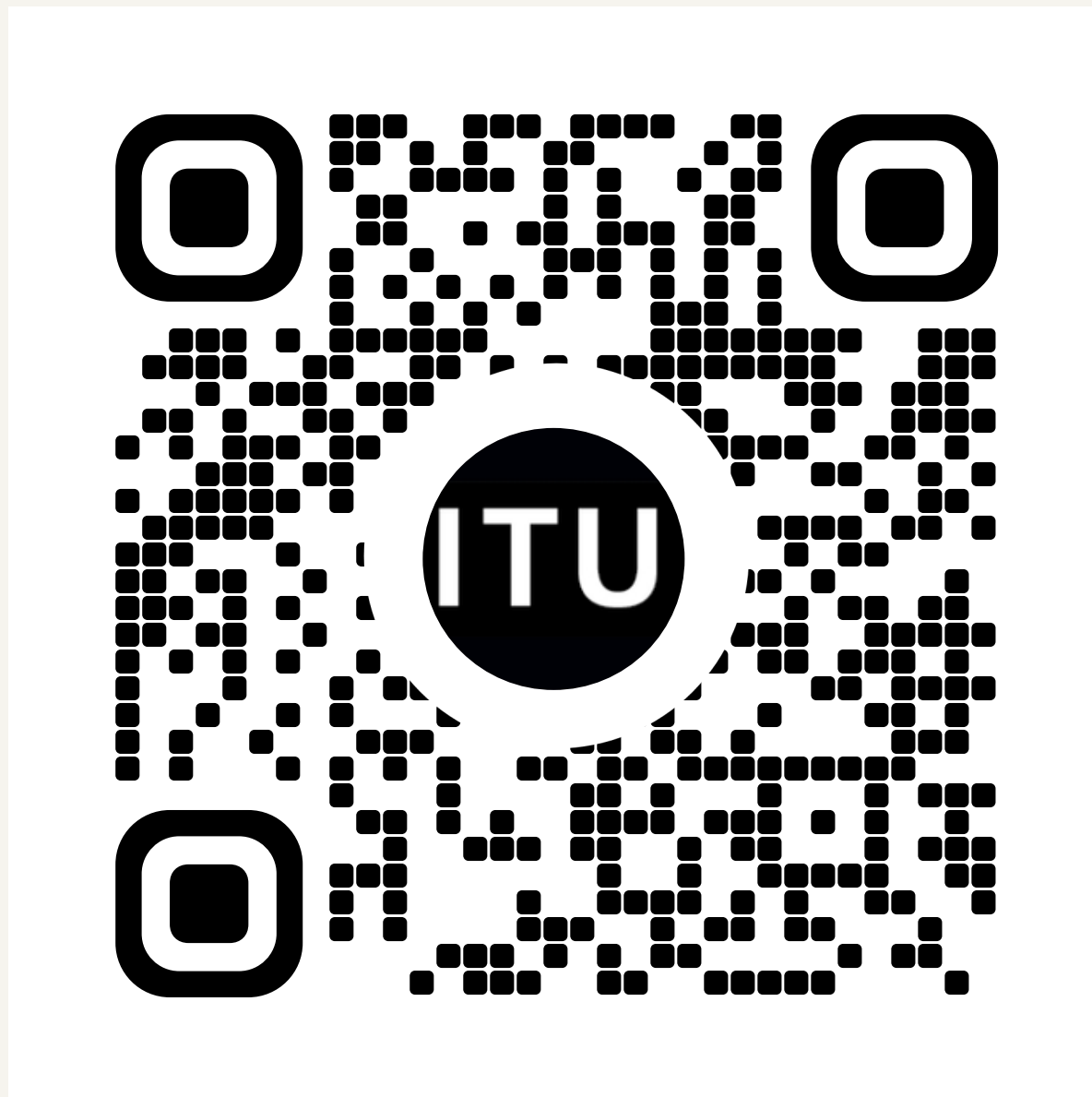


```
Kart Configuration:  
- Body: Badwagon  
- Tire: Wood  
- Glider: Cloud
```

BootCode - Online IDE Built for BootIT

Again! Access the website via a laptop by...

Scanning the QR code



Join today's session, refresh if you don't see it.

Entering the web address directly

BootCode - ITU BootIT

BootCode is the beginner-friendly Java coding platform for ITU BootIT, the pre-master computer science bootcamp. Write, run, and submit Java exercises directly in your browser.

cobblerscoders.tech

<https://cobblerscoders.tech/>

Clicking the link on the course platform



<https://learnit.itu.dk/course/view.php?id=3026730>

Exercise - Create a Person Class

36

Create a **Person** class that meets the following requirements:

- Fields:
 - **name** (String)
 - **age** (int)
- Constructor:
 - **Person**(String n, int a): Initializes the name and age fields.
- Methods:
 - **display**(): Prints the person's information in the exact format: "[name] ([age] years old)". (Example output: "Niek (25 years old)")
 - **birthday**(): Increases the person's age by 1.

Exercise - Create a FlightTicket Class

37

Create a **FlightTicket** class that meets the following requirements:

- Fields:
 - **from** (String)
 - **to** (String)
 - **price** (int or double)
- Constructor:
 - **FlightTicket(String f, String t, int p)**: Initializes the from, to, and price fields.
- Methods:
 - **show()**: Prints the ticket information in the exact format: "[from] --> [to] ([price] DKK)". (Example output: "CPH --> JFK (7500 DKK)")
 - **discount()**: Reduces the ticket price by 500. You must add logic to ensure this method cannot be abused (i.e., the price must never drop below 0).

Exercise - Create a Container Class

38

Create a **Container** class that meets the following requirements:

- Fields:
 - **id** (String)
 - **amount** (int)
 - **max** (int)
- Constructor:
 - **Container(String i, int max)**: Initializes the id and max fields. The amount field must always start at 0.
- Methods:
 - **show()**: Prints the container's information in the exact format: "Container: [id] ([amount]/[max])". (Example output: "Container: AX35 (23/30)")
 - **addCargo(int a)**: Adds the specified number of boxes (a) to the container's amount.
 - **Capacity Constraint**: The container cannot be over-filled. If adding a would cause the amount to exceed max, the addition must be rejected entirely.

(Mini) Projects



Pick a project to start with!

Choose **any** of the 4 options and complete as many as you'd like.

Please write and test your code in VS Code, then drag and drop your files to BootCode to submit.

Build' a Tree®

Grandpa's
TIME MACHINE

**Grandma's Blackmarket
Kitchen**

SEAT SELECTOR